

Laboratorium Ilmu Rekayasa dan Komputasi

TEKNIK INFORMATIKA
STEI ITB

IF2224 - Formal Language and
Automata Theory

Arion Interpreter

Milestone 4 & 5

Intermediate Code Generator & Interpreter

Milestone 4

Intermediate Code and Interpreter

Release : Kamis, 21 Mei 2026

Deadline : Kamis, 4 Juni 2026 pukul 23.59 WIB

Revisi : -

Daftar Revisi

[DD-MM-YY] Revisi X. Isi Revisi

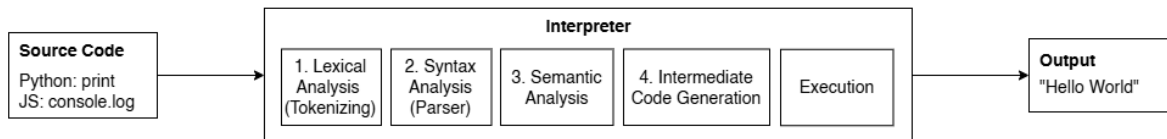
Daftar Isi

Daftar Revisi.....	2
Daftar Isi.....	4
I. Pendahuluan.....	5
II. Teori Singkat.....	6
III. Spesifikasi Tugas Besar.....	7
IV. Teknis Pengerjaan.....	11
V. Pengumpulan.....	13
Referensi.....	14
Lampiran.....	15

I. Pendahuluan

Selamat, Anda telah mencapai tahap akhir dari pengembangan bahasa pemrograman Arion!

High Level - Interpreted Language



Gambar 1. Proses *Interpreted Language* dari Tokenisasi hingga Eksekusi
(Sumber: asisten)

Mari kita kilas balis sejenak perjalanan *source code* yang ditulis oleh programmer hingga titik ini sesuai Gambar 1. Pada Milestone I (Lexical Analysis), kode program yang berupa teks mentah telah dipecah menjadi kepingan bermakna yang disebut *token*. Kemudian pada Milestone II (Syntax Analysis), kumpulan token tersebut disusun ke dalam struktur hierarkis berupa *Parse Tree* untuk memastikan urutannya sesuai dengan *grammar*. Selanjutnya pada Milestone III (Semantic Analysis), struktur tersebut divalidasi kebenaran logika dan tipe datanya.

Kini, kita memiliki representasi program yang sudah 100% valid secara sintaksis maupun semantik. Pertanyaannya: **Apakah kita sudah bisa menjalankan program tersebut?**

Secara teoritis, kita bisa saja membuat program yang langsung menelusuri *Parse Tree* dari atas ke bawah untuk mengeksekusinya. Namun, dalam dunia nyata, cara ini sangat tidak efisien. Memori komputer dan CPU dirancang untuk mengeksekusi instruksi secara linier atau berurutan (baris demi baris), bukan dengan melompat-lompat di dalam struktur pohon yang kompleks.

```

Decorated AST (contoh anotasi minimal):
ProgramNode(name: 'Hello')
├── Declarations
│   ├── VarDecl('a')      → tab_index:30, type:integer, lev:0
│   └── VarDecl('b')      → tab_index:31, type:integer, lev:0
├── Block                  → block_index:1, lev:1
│   ├── Assign('a' := 5)  → type:void
│   │   ├── target 'a'    → tab_index:30, type:integer
│   │   └── value 5        → type:integer
│   ├── Assign('b' := a+10) → type:void
│   │   ├── target 'b'    → tab_index:31, type:integer
│   │   └── BinOp '+'      → type:integer
│   │       ├── 'a'        → tab_index:30, type:integer
│   │       └── 10          → type:integer
│   └── writeln(...)      → predefined, tab_index:32

```

Gambar 2. Contoh Decorated AST

```

0 INT 0 5
1 LIT 0 5
2 STO 0 3
3 LOD 0 3
4 LIT 0 10
5 OPR 0 2
6 STO 0 4
7 LOD 0 4
8 OPR 0 14
9 RET 0 0

```

Gambar 3. Contoh Intermediate Code (;' memulai komentar)

Oleh karena itu, pada Milestone IV ini, kita membutuhkan jembatan terakhir sebelum kode benar-benar dieksekusi, yaitu **Intermediate Code Generation**. Pada tahap ini, Anda akan “meratakan” bentuk pohon yang rumit tadi menjadi daftar instruksi sekuensial yang sangat sederhana dan mirip dengan bahasa mesin, namun masih cukup mudah dibaca oleh manusia. Bentuk perantara inilah yang disebut sebagai *Intermediate Code*.

Setelah daftar instruksi perantara ini berhasil dibuat, tokoh utama terakhir kita akan mengambil alih: **Interpreter**. Interpreter bertindak layaknya sebuah *virtual machine*. Ia akan membaca *Intermediate Code* tersebut baris demi baris, menyimpan nilai variabel ke dalam memori, mengevaluasi operasi matematika, mengatur ke mana arah jalannya program (jika ada percabangan seperti **IF** atau perulangan **WHILE**), dan pada akhirnya mencetak hasil nyata ke layar komputer.

Merujuk kembali pada Gambar 1, milestone ini adalah puncak dari Tugas Besar Anda. Semua komponen yang telah Anda bangun secara terpisah pada milestone-milestone sebelumnya akan disatukan. Di akhir tahap ini, bahasa pemrograman Arion tidak lagi sekadar memproses teks, tetapi sudah “hidup” dan mampu memecahkan masalah komputasi layaknya bahasa pemrograman sungguhan seperti Python atau JavaScript.

II. Teori Singkat

A. Pengantar Teori

Dalam perancangan *compiler* dan *interpreter* modern, sistem umumnya dibagi menjadi dua fase arsitektur utama: fase **Front-end** dan fase **Back-end**.

Milestone I hingga III yang telah Anda selesaikan merupakan bagian dari *Front-end*. Tugas utama *Front-end* adalah berurusan langsung dengan *source code* yang ditulis oleh pengguna: memotong teksnya, memvalidasi tata bahasanya, dan memastikan maknanya benar. Hasil akhir dari *Front-end* adalah struktur data murni (seperti *Decorated AST*) yang sudah terbebas dari teks mentah.

Pada Milestone IV ini, kita sepenuhnya memasuki area *Back-end*. Fase *Back-end* sama sekali tidak peduli dengan sintaksis bahasa Arion (apakah bahasa tersebut menggunakan titik koma, kurung kurawal, atau spasi). Tugas *Back-end* berfokus murni pada **Sintesis dan Komputasi**:

1. **Sintesis:** Mengonversi *Decorated AST* menjadi instruksi operasional standar (*Intermediate Code*)
2. **Eksekusi:** Mengelola memori dan menjalankan instruksi tersebut pada mesin virtual (*Interpreter*)

Pemisahan arsitektur yang tegas antara *Front-end* dan *Back-end* ini sangat penting. Dengan desain ini, jika suatu saat Anda ingin membuat bahasa Arion versi baru dengan sintaks yang berbeda, Anda hanya perlu merombak *Front-end*-nya saja. Selama *Front-end* baru tersebut menghasilkan *Decorated AST* yang sama, mesin *Back-end* Anda akan tetap bisa mengeksekusinya tanpa perlu diubah sedikit pun.

B. Intermediate Code Generation & Decorated AST

Setelah *Front-end* selesai memvalidasi program, tugas pertama *Back-end* adalah menerjemahkan pemahaman konseptual tersebut ke dalam bentuk instruksi yang siap dieksekusi. Proses ini tidak bisa dilakukan secara sembarangan; ia membutuhkan sumber kebenaran yang mutlak, yaitu *Decorated AST*.

a. Decorated AST

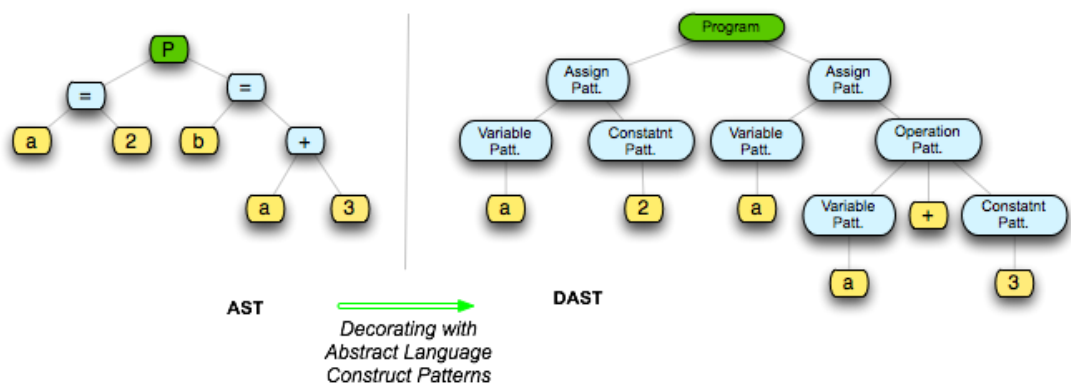
Pada akhir Milestone III (Semantic Analysis), *Parse Tree* atau *AST* anda tidak lagi sekadar merepresentasikan struktur teks, melainkan telah “didekorasi” atau dianotasi dengan informasi semantik. Inilah yang disebut dengan ***Decorated AST***.

Informasi tambahan yang melekat pada setiap *node* AST ini umumnya meliputi:

1. **Tipe Data:** Apakah ekspresi pada *node* ini akan menghasilkan tipe data A, B, C, atau D?
2. **Referensi Tabel Simbol:** Jika *node* tersebut adalah sebuah variabel (misalnya *x*), ia harus memiliki *pointer* atau referensi langsung ke entri *x* di dalam Tabel Simbol untuk mengetahui ruang lingkungannya dan letak memorinya.

Mengapa Decorated AST berstatus wajib?

Bayangkan generator Anda menemui ekspresi “a + b” di dalam AST. Tanpa anotasi tipe data, generator tidak tahu apakah ini adalah instruksi matematika (penjumlahan angka) atau instruksi manipulasi teks (penggabungan string). Dengan *Decorated AST*, generator dapat dengan cerdas memutuskan instruksi *Intermediate Code* spesifik apa yang harus dihasilkan.



Gambar 4. Ilustrasi Perbandingan *Plain AST* dengan *Decorated AST*
(Sumber: https://epl.di.uminho.pt/~gepl/GEPL_DS/Alma2/dapast.php)

b. Three-Adress Code

Langkah selanjutnya adalah meratakan (*flattening*) *Decorated AST* tersebut menjadi *Intermediate Code*. Dari berbagai macam format yang ada, format standar yang paling luas digunakan di industri adalah **Three-Adress Code (TAC)**.

Dinamakan “Three-Adress” karena setiap baris instruksi didesain untuk sangat sederhana, di mana maksimal hanya boleh melibatkan **tiga alamat memori** (biasanya terdiri dari dua *operand* sumber dan satu lokasi tujuan).

Struktur dasar TAC terlihat seperti ini:

```
hasil = operand1 operator operand2
```

Penggunaan Variabel Sementara

Dalam bahasa tingkat tinggi, programmer bebas menulis ekspresi matematis yang sangat panjang dan kompleks. Namun, CPU tidak bisa menghitung semuanya sekaligus. CPU harus memecahnya menjadi operasi-operasi biner yang sederhana. TAC menyimulasikan batasan CPU ini dengan menggunakan **Variabel Sementara** (biasanya direpresentasikan dengan t1, t2, t3, dst.).

Mari kita bedah proses translasinya. Misalkan terdapat *source code* Arion berikut:

```
x := (a + b) * (c - d)
```

Secara internal, AST dari ekspresi di atas berbentuk pohon bersarang. TAC akan meratakannya dengan cara mengevaluasi dari cabang terdalam (paling bawah di AST) bergerak ke atas (akar), menghasilkan instruksi linier berikut:

```
t1 := a + b      { Langkah 1: Evaluasi kurung pertama }
t2 := c - d      { Langkah 2: Evaluasi kurun kedua   }
t3 := t1 * t2    { Langkah 3: Kalikan hasil keduanya }
x  := t3         { Langkah 4: Simpan hasil akhir     }
```

Dengan memecah ekspresi menggunakan TAC, kompleksitas kode tingkat tinggi telah berhasil diturunkan menjadi barisan instruksi primitif yang sangat mudah dieksekusi oleh mesin.

c. Translasi Control Flow

Mengubah ekspresi matematika menjadi TAC relatif mudah karena polanya pasti. Tantangan terbesar dalam *Intermediate Code Generation* adalah meratakan *Control Flow* seperti `IF-ELSE` dan `WHILE`.

Bagaimana caranya kita meratakan struktur blok kode yang bersarang menjadi daftar linier? Jawabannya adalah dengan menggunakan kombinasi **Label** dan **Jump (GOTO)**.

1. **Label:** Sebuah penanda statis yang menunjukkan posisi baris tertentu dalam TAC (misal: `L1:`, `LABEL_END:`)
2. **Jump Bersyarat (`IF_FALSE ... GOTO ...`):** Melompat ke suatu Label hanya jika kondisi yang dievaluasi bernilai *False*.
3. **Jump Mutlak (`GOTO ...`):** Melompat secara langsung ke suatu Label tanpa memedulikan kondisi apa pun.

Translasi IF-ELSE

Mari kita lihat bagaimana logika percabangan diratakan

Source Code Arion

```
if x > 10 then
    y := 1
else
    y := 0;
```

Hasil TAC

```
t1 := x > 10
IF_FALSE t1 GOTO L_ELSE
y := 1
GOTO L_END
L_ELSE:
    y := 0
L_END:
```

Translasi Perulangan (WHILE Loop)

Perulangan memiliki kerumitan ekstra karena setelah blok kode selesai dieksekusi, alur program harus “melompat mundur” ke atas untuk mengevaluasi ulang kondisinya

Source Code Arion

```
while i < 5 do
begin
    i := i + 1;
end;
```

Hasil TAC

```
L_START:
    t1 := i < 5
    IF_FALSE t1 GOTO L_END

    t2 := i + 1
    i := t2

    GOTO L_START
L_END:
```

Melalui pemetaan *Label* dan *Jump* ini, Anda telah berhasil menghilangkan kebutuhan akan blok bersarang. *Interpreter* Anda nantinya tidak perlu tahu apa itu “perulangan”; ia hanya perlu dengan patuh menjalankan instruksi membaca, mengevaluasi, dan melompat sesuai dengan arahan kode TAC.

Mengapa kita menggunakan IF_FALSE dan bukan IF_TRUE ?

Penggunaan IF_FALSE dibandingkan IF_TRUE dalam TAC sering menimbulkan pertanyaan, namun sebenarnya keduanya sama-sama valid dan tidak ada aturan baku yang mewajibkan salah satunya. Pada level bahasa mesin (misalnya assembly), tersedia instruksi untuk melompat baik saat kondisi bernilai benar maupun salah, sehingga tidak ada preferensi mutlak.

Alasan `IF_FALSE` sering digunakan adalah karena memanfaatkan konsep ***fall-through***, yaitu ketika kondisi bernilai benar, eksekusi dapat langsung berlanjut ke instruksi berikutnya tanpa perlu lompatan, sementara lompatan hanya terjadi jika kondisi salah. Pendekatan ini dapat menyederhanakan alur kontrol dan dalam beberapa kasus lebih efisien. Meski demikian, *compiler* modern tidak selalu menggunakan `IF_FALSE`, karena pilihan tersebut bergantung pada berbagai faktor seperti optimasi, tata letak kode, dan prediksi percabangan. Oleh karena itu, `IF_FALSE` hanyalah salah satu strategi yang umum digunakan, bukan standar yang wajib diikuti (**tetapi pada tugas besar ini wajib**).

C. Arsitektur Mesin Eksekusi: Interpreter & The Stack

Setelah *Intermediate Code Generator* selesai meratakan *Decorated AST* menjadi daftar instruksi TAC, pekerjaan translasi bahasa telah usai. Kini saatnya program Anda benar-benar “hidup”. Tugas ini diembang oleh **Interpreter**, yang pada dasarnya bertindak sebagai sebuah **Virtual Machine (VM)**. VM buatan Anda ini tidak memiliki bentuk fisik, tetapi ia memiliki logika memori dan siklus kerja yang sama persis.

a. Konsep Virtual Machine dan Siklus Eksekusi

Sama seperti CPU sungguhan, Interpreter Anda harus membaca instruksi satu per satu. Untuk melacak baris instruksi yang sedang dibaca, Interpreter menggunakan sebuah penunjuk khusus yang disebut **Instruction Pointer (IP)** atau *Program Counter (PC)*. Anda bisa membayangkan IP sebagai jari Anda yang menunjuk baris demi baris saat membaca buku.

Interpreter beroperasi dalam sebuah **Siklus Eksekusi**, yang terdiri dari tiga tahap utama:

1. **Fetch:** Membaca baris instruksi TAC yang saat ini ditunjuk oleh IP.
2. **Decode:** Membedah baris tersebut untuk mengetahui apa perintahnya (misal: Apakah ini operasi penambahan `+`? Apakah ini perintah `GOTO`? Siapa saja *operand*-nya?)
3. **Execute:** Melakukan aksi nyata. Jika itu adalah operasi matematika, maka VM akan menghitung nilainya. Jika itu adalah perintah assignment (`x := 5`), VM akan menyimpannya ke memori.

Setelah tahap *Execute* selesai, IP akan bergerak maju ke baris instruksi berikutnya ($IP := IP + 1$), kecuali jika instruksi tersebut adalah *GOTO*, di mana IP akan langsung melompat ke nomor baris yang dituju oleh *Label*. Siklus ini terus berulang hingga IP mencapai akhir dari daftar instruksi TAC.

b. Memori Eksekusi: The Stack

Untuk bisa melakukan perhitungan dan menyimpan variabel, VM Anda membutuhkan memori. Dalam arsitektur *interpreter*, struktur memori yang paling vital adalah **The Stack**.

Stack bekerja dengan prinsip **LIFO (Last In, First Out)**. Bayangkan *Stack* seperti tumpukan piring di restoran; Anda hanya bisa menaruh piring baru di tumpukan paling atas (*Push*), dan Anda hanya bisa mengambil piring dari tumpukan paling atas (*Pop*).

Mengapa kita menggunakan *Stack* dan bukan sekadar *array* biasa? Jawabannya berkaitan erat dengan **Scope** dan **Function Call**.

Setiap kali program Anda memasuki sebuah blok kode baru atau memanggil sebuah fungsi, *Interpreter* akan membuat sebuah kotak memori baru di dalam *Stack* yang disebut *Stack Frame*.

1. **Isi Stack Frame:** Di dalam *frame* inilah *Interpreter* menyimpan semua variabel lokal milik fungsi tersebut, argumen yang di-*passing*, serta *return address*.
2. **Push & Pop:** Saat fungsi dipanggil, *Stack Frame* barunya di-*push* ke pucuk tumpukan. VM hanya boleh membaca dan mengubah variabel yang berada di *frame* paling atas ini. Ketika fungsi selesai dieksekusi, *frame* tersebut di-*pop* dari *Stack*, dan semua variabel lokalnya otomatis musnah. VM kemudian kembali fokus pada *frame* di bawahnya.

Mekanisme *Stack Frame* inilah yang menjelaskan mengapa variabel lokal di dalam fungsi A tidak bisa diakses oleh fungsi B, dan mengapa memori program bisa tetap hemat.

c. Runtime Type Checking

Anda mungkin bertanya: “Jika pada Milestone III (Semantic Analysis) kita sudah mengecek tipe datanya, mengapa *Interpreter* masih perlu peduli soal tipe data?”

Semantic Analysis melakukan pengecekan secara statis (sebelum program berjalan). Ia mengecek aturan. Namun, *Interpreter* berhadapan dengan nilai aktual saat program *runtime*.

Dalam bahasa pemrograman dinamis atau saat berhadapan dengan konversi tipe data implisit, nilai yang ada di dalam variabel bisa berubah wujud atau memicu ambiguitas saat dieksekusi. Di sinilah **Runtime Type Checking** menggunakan informasi dari **Decorated AST** menjadi syarat wajib pada *interpreter*. Anda jika bahasa pemrograman bersifat *weakly-typed* (akan tetapi, **bahasa Arion adalah *strongly-typed* sehingga tidak diwajibkan adanya *runtime type checking***).

1. Saat *Interpreter* melakukan tahap *Execute* pada instruksi TAC (misalnya $t3 := t1 + t2$), ia akan melihat kembali informasi tipe data dari Decorated AST.
2. *Interpreter* akan memvalidasi tipe data dari nilai aktual yang ada di dalam memori (*Stack*).
3. Jika *Decorated AST* mengatakan ini adalah operasi `Integer + Integer`, tetapi saat *runtime* memori entah bagaimana membaca $t2$ sebagai `String` (mungkin karena masukan pengguna yang tidak terduga atau kesalahan konversi), *Interpreter* harus dengan sigap menghentikan eksekusi dan melempar **Runtime Error**.

D. [Bonus] Vulnerabilities pada Interpreter

Membangun sebuah *Interpreter* yang bisa menjalankan kode dengan benar adalah sebuah pencapaian besar. Namun, *Interpreter* yang hebat tidak hanya tahu cara menjalankan kode yang benar, tetapi juga tahu **bagaimana cara bertahan hidup (tidak *crash*)** ketika diberikan kode yang salah, berbahaya, atau tidak masuk akal.

Bahasa pemrograman tingkat rendah seperti C dan C++ mendelegasikan tanggung jawab keamanan ini kepada *programmer*. Jika *programmer* melakukan kesalahan, program akan langsung *crash* di tingkat OS (misalnya

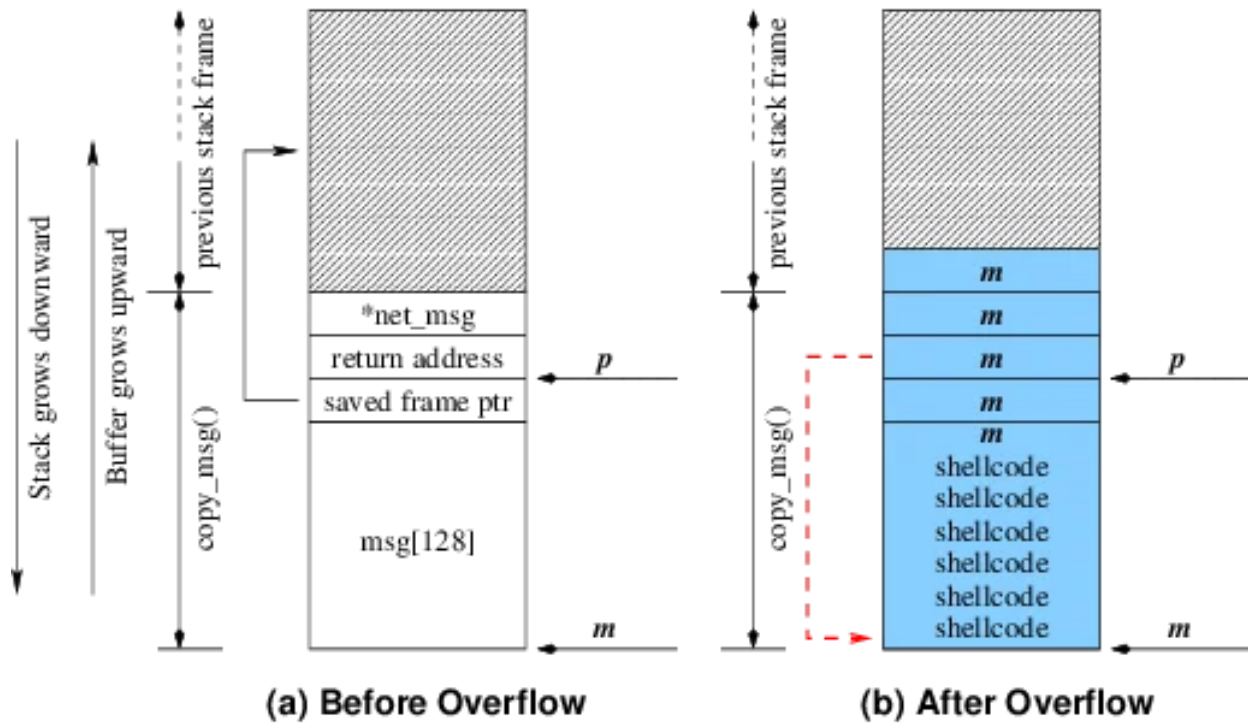
muncul *segmentation fault*). Namun, bahasa tingkat tinggi yang berjalan di atas VM (seperti Java, Python, dan **Arion** yang sedang Anda buat) harus memiliki mekanisme perlindungan internal.

Sebagai implementasi bonus pada Milestone IV ini, Anda ditantang untuk menambahkan deteksi dan penanganan terhadap kerentanan berikut di dalam *Interpreter* Anda.

a. Kerentanan pada Stack

Seperti yang dibahas sebelumnya, *Stack* adalah nyawa dari eksekusi memori. Jika *Stack* rusak, seluruh program akan runtuh.

1. **Stack Overflow:** Terjadi ketika program terus-menerus menambahkan (*Push*) *Stack Frame* baru hingga melampaui batas memori yang dialokasikan oleh *Interpreter*. Kasus paling umum yang memicu ini adalah ***infinite recursion***, di mana sebuah fungsi terus memanggil dirinya sendiri tanpa memiliki *base case*. *Interpreter* Anda harus mampu menghitung kedalaman *Stack* dan melempar *error* jika melebihi batas (misal: maksimal 1000 *frame*).
2. **Stack Underflow:** Kebalikan dari *Overflow*. Ini terjadi ketika *Interpreter* diinstruksikan untuk mengambil (*Pop*) data dari *Stack* yang sudah sepenuhnya kosong. Ini biasanya menandakan adanya cacat logika pada *Intermediate Code Generator* Anda (misalnya: mencoba me-*return* nilai dari *scope* global).
3. **Stack Smashing:** Ini adalah bentuk klasik dari *Buffer Overflow*. Dalam eksekusi fungsi, memori lokal diletakkan bersebelahan dengan *return address*. Jika *Interpreter* mengizinkan operasi yang menulis data melebihi ukuran variabel lokalnya, data tersebut bisa “meluber” dan menimpa *return address*. Akibatnya, saat fungsi selesai, ia melompat ke memori yang salah (atau bahkan mengeksekusi kode berbahaya).
4. **Stack Corruption:** Merupakan istilah umum ketika struktur *Stack* kehilangan keseimbangannya. Jumlah operasi *Push* dan *Pop* harus selalu simetris. Jika sebuah blok kode melakukan 3 kali *Push* tapi hanya 2 kali *Pop* sebelum keluar, akan ada “sampah” memori yang tertinggal dan menggeser posisi indeks *Stack Frame*, menyebabkan seluruh pembacaan variabel selanjutnya menjadi salah.



Gambar 5. Ilustrasi Stack Smashing: merusak struktur krusial Stack

b. Kerentanan Memori dan Akses: Out-of-Bounds Variable Access

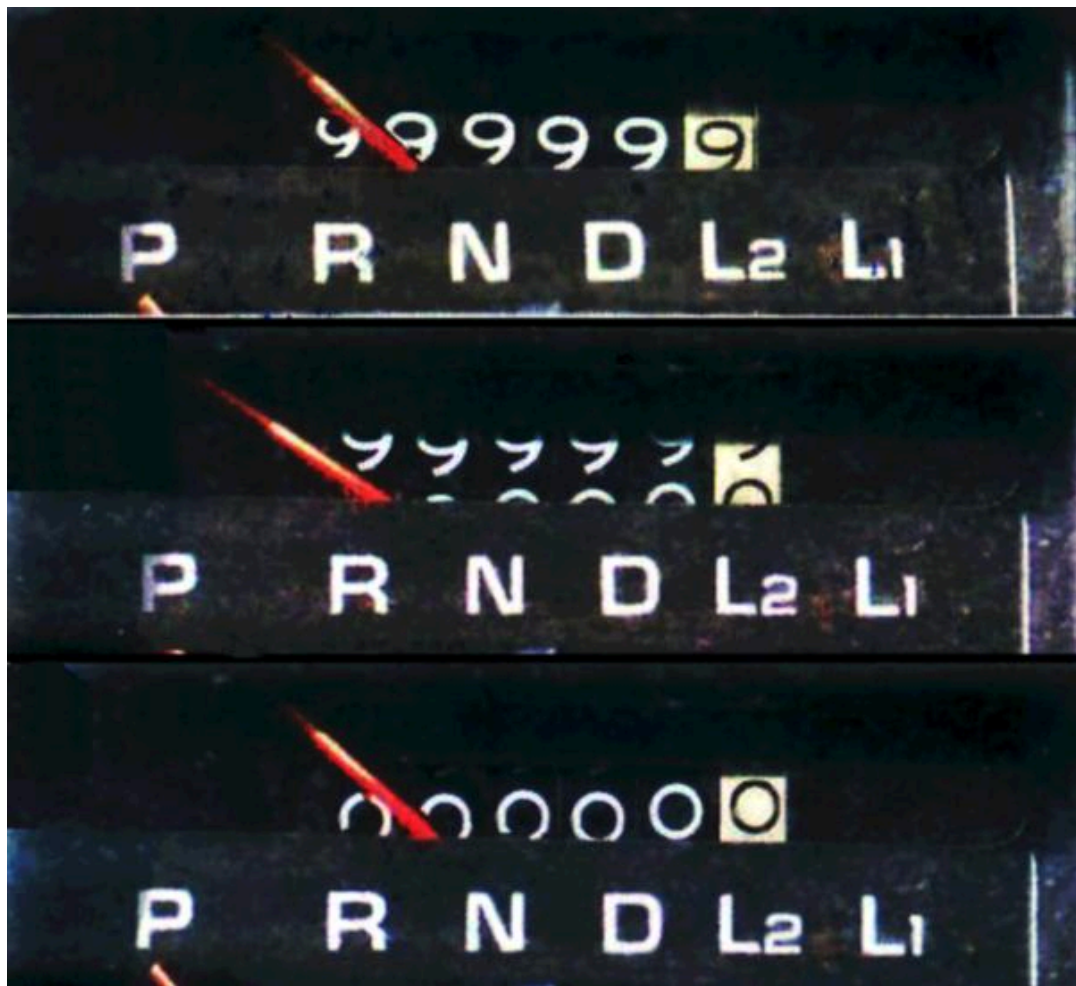
Salah satu jenis kerentanan yang umum terjadi adalah **Out-of-Bounds Variable Access**. Karena bahasa Arion mendukung struktur data seperti Array atau List, *Interpreter* wajib memvalidasi setiap akses indeksnya. Jika *array* memiliki panjang 5 (dengan indeks 0-4), dan program mencoba mengakses indeks ke-10 atau indeks -1, *Interpreter* tidak boleh meneruskan instruksi tersebut ke memori fisik komputer, melainkan harus mencegatnya dan melempar **IndexOutOfBoundsException**.

c. Kerentanan Alur Eksekusi: Invalid Jump Targets

Seperti yang kita tahu, instruksi *Control Flow* diubah menjadi TAC berupa `GOTO Label`. Apa yang terjadi jika *Intermediate Code Generator* melakukan kesalahan dan menyuruh *Interpreter* melompat ke `LABEL_X`, tetapi `LABEL_X` tersebut tidak pernah ada di dalam daftar instruksi? Atau bagaimana jika ia melompat ke tengah-tengah instruksi yang bukan merupakan batas operasi? *Interpreter* yang tangguh harus memverifikasi semua target *Jump* sebelum atau saat dieksekusi agar IP tidak tersesat ke area memori yang acak.

d. Kerentanan Aritmatika: Numerical Overflow/Underflow

Tipe data di dalam komputer memiliki kapasitas maksimum. Misalnya, sebuah *signed Integer* 32-bit hanya bisa menampung nilai maksimal sekitar 2,14 miliar. Jika nilai ini ditambah 1, di tingkat perangkat keras nilainya tidak menjadi 2,14 miliar plus satu, melainkan terjadi *wrap-around* menjadi bilangan -2,14 miliar. Jika *Interpreter* Anda tidak bisa mendeteksi limit ini, perhitungan finansial atau logika perulangan di dalam program bisa menjadi sangat kacau. *Interpreter* harus mampu mendeteksi operasi yang akan melampaui batas (*overflow*) atau jatuh di bawah batas minimum (*underflow*), lalu menghentikan program dengan **OverflowError** atau **UnderflowError**.



Gambar 6. Ilustrasi odometer mobil rollover “Analogi Numerical Overflow: Ketika nilai melampaui kapasitas penyimpanan, ia tergulung kembali”

III. Spesifikasi Tugas Besar

Pada milestone ini, Anda diminta untuk membuat dua komponen yaitu Intermediate Code Generator, *Stack* Penyimpanan, dan Interpreter. Intermediate Code Generator berguna untuk mengubah decorated AST menjadi sebuah langkah-langkah yang dibutuhkan untuk menjalankan program. Informasi-informasi ini akan disimpan dalam *stack* penyimpanan dan akan digunakan ketika ada operasi tertentu. Lalu berdasarkan kode tersebut, bilamana ada `writeln`, akan keluar hasil akhir dari kode yang dibuat.

Tahapan ini bertujuan untuk mengubah decorated AST menjadi output final dari sebuah kode Arion. Semua tahapan-tahapan sudah membantu anda untuk sampai ke titik ini. Tahapan ini juga bertujuan untuk mendapatkan pemahaman mengenai bagaimana mesin mengelola memori yang berisi informasi-informasi penting seperti variabel, fungsi, prosedur, dll.

Masukan	Decorated AST
Keluaran	Intermediate Code dan output sebenarnya (Jika ada <code>writeln</code>)
Algoritma	Intermediate Code Generation: Depth First Search based approach (Menggunakan prinsip DFS tetapi bukan DFS murni) Interpreter: Stack Machine

Berikut merupakan contoh daftar tipe instruksi, operasi, dan penjelasan dari instruksi tersebut. Ini adalah instruksi-instruksi yang digunakan untuk membuat Intermediate Code. Pada saat mencapai tahap Interpreter, instruksi akan dijalankan pada operasi-operasi yang ada di sebelah kanan instruksi tersebut.

Daftar Instruksi			
No	Instruksi	Operasi	Keterangan
1	LIT v	Load Literal	Memasukkan nilai literal v ke dalam stack
2	LOD a	Load Value	Memuat nilai dari address a
3	STO a	Store Value	Menyimpan nilai ke address a

4	CAL I	Call	Memanggil fungsi di garis l
5	INT m	Initiate Memory	Membuat memory dengan ukuran m
6	JMP I	Unconditional Jump	Lompat ke garis l tanpa kondisi apapun
7	JPC I	Conditional Jump	Lompat ke garis l bila kondisi tidak memenuhi
8	OPR o	Operation	Memanggil operasi o (Termasuk fungsi bawaan Arion)
9	RET	Return	Keluar dari fungsi atau prosedur

Berikut juga operasi-operasi yang bisa dijalankan oleh instruksi 'OPR'

Instruksi OPR (Jalankan seperti ini OPR [value_hierarchy] [operation_no])		
No	Operation Name	Function
1	NEG	Negasi value dari stack teratas
2	ADD	Menambah value stack teratas dengan value stack teratas satunya lagi
3	SUB	Mengurangi value stack teratas dengan value stack teratas satunya lagi
4	MUL	Mengali value stack teratas dengan value stack teratas satunya lagi
5	DIV	Membagi value stack teratas dengan value stack teratas satunya lagi
6	MOD	Modulus value stack teratas dengan value stack teratas satunya lagi
7	EQL	Membandingkan kedua value teratas dari stack untuk melihat apakah kedua value tersebut sama (Conditionals)
8	NEQ	Membandingkan kedua value teratas dari stack untuk melihat apakah kedua value tersebut tidak sama (Conditionals)

9	LSS	Membandingkan kedua value teratas dari stack untuk melihat apakah value pertama kurang dari value kedua (Conditionals)
10	GEQ	Membandingkan kedua value teratas dari stack untuk melihat apakah value pertama lebih dari sama dengan value kedua (Conditionals)
11	GTR	Membandingkan kedua value teratas dari stack untuk melihat apakah value pertama lebih dari value kedua (Conditionals)
12	LEQ	Membandingkan kedua value teratas dari stack untuk melihat apakah value pertama kurang dari sama dengan value kedua (Conditionals)
13	WRT	Menulis output yang sudah dimuat
14	WRTLN	Menulis output yang sudah dimuat lalu diberikan newline

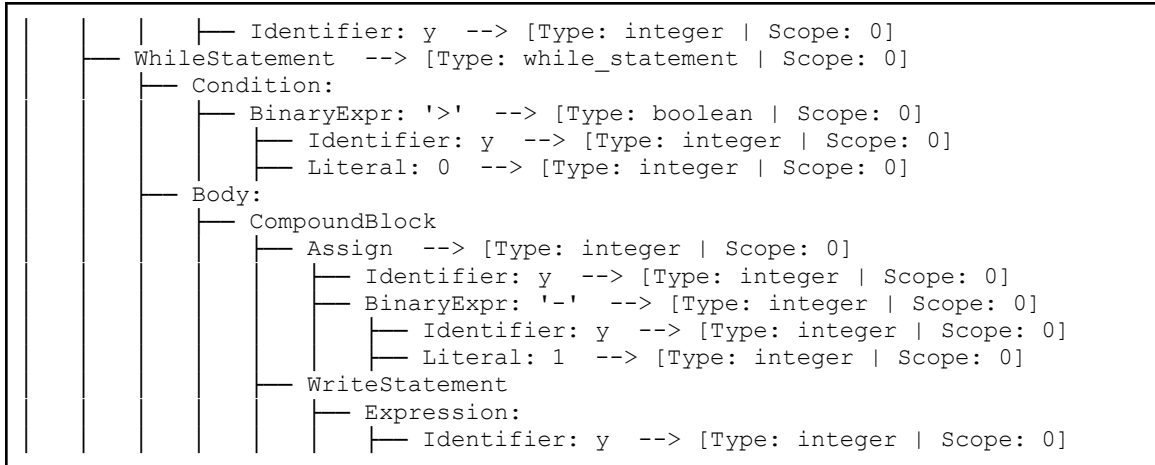
Berikut adalah step-by-step cara mengubah decorated AST menjadi sebuah intermediate code:

1. Contoh Decorated AST

```

Program: TestKeywords --> [Type: program | Scope: 0]
├── VarDecl: x : integer --> [Type: integer | Scope: 0]
├── VarDecl: y : integer --> [Type: integer | Scope: 0]
├── Assign --> [Type: integer | Scope: 0]
│   ├── Identifier: x --> [Type: integer | Scope: 0]
│   └── Literal: 10 --> [Type: integer | Scope: 0]
├── IfStatement --> [Type: if_statement | Scope: 0]
│   ├── Condition:
│   │   ├── BinaryExpr: '>' --> [Type: boolean | Scope: 0]
│   │   │   ├── Identifier: x --> [Type: integer | Scope: 0]
│   │   │   └── Literal: 5 --> [Type: integer | Scope: 0]
│   │   └── Then:
│   │       ├── Assign --> [Type: integer | Scope: 0]
│   │       │   ├── Identifier: y --> [Type: integer | Scope: 0]
│   │       │   └── BinaryExpr: '*' --> [Type: integer | Scope: 0]
│   │       │       ├── Identifier: x --> [Type: integer | Scope: 0]
│   │       │       └── Literal: 2 --> [Type: integer | Scope: 0]
│   │       └── Else:
│   │           ├── Assign --> [Type: integer | Scope: 0]
│   │           │   ├── Identifier: y --> [Type: integer | Scope: 0]
│   │           │   └── BinaryExpr: 'div' --> [Type: integer | Scope: 0]
│   │           │       ├── Identifier: x --> [Type: integer | Scope: 0]
│   │           │       └── Literal: 2 --> [Type: integer | Scope: 0]
│   └── WriteStatement
│       └── Expression:

```



2. Definisi Memori

Karena tidak ada prosedur atau fungsi lainnya, akan hanya ada satu inisiasi memori saja. Perhatikan bagian ini

```

Program: TestKeywords --> [Type: program | Scope: 0]
├── VarDecl: x : integer --> [Type: integer | Scope: 0]
└── VarDecl: y : integer --> [Type: integer | Scope: 0]
  
```

Pada program TestKeywords. Akan dilakukan pendefinisian dua variabel yaitu 'x' dan 'y'. Terdapat bagian dari memori yang sudah akan selalu diisi oleh Static link, Dynamic link, dan Return address. Sisanya bebas digunakan oleh sistem dan biasanya digunakan untuk variabel. Berdasarkan tabel instruksi sebelumnya, sistem intermediate code anda perlu mengembangkan instruksi untuk menginisiasi memori sebanyak 5. Hasilnya adalah seperti ini.

```
0 INT 0 5
```

3. Assign 10 ke x

```

├── Assign --> [Type: integer | Scope: 0]
│   ├── Identifier: x --> [Type: integer | Scope: 0]
│   └── Literal: 10 --> [Type: integer | Scope: 0]
  
```

Melihat instruksi di atas artinya sistem perlu mendefinisikan nilai literal yaitu 10 lalu menyimpannya ke dalam variabel x. Kita tahu bahwa x berada di address 3 sehingga sistem akan mengembangkan perintah untuk memanggil nilai literal 10 lalu menyimpannya di address 3.

```
1  LIT  0  10
2  STO  0  3
```

4. IF statement - Operasi perbandingan

```

├─ IfStatement --> [Type: if_statement | Scope: 0]
│   └─ Condition:
│       └─ BinaryExpr: '>' --> [Type: boolean | Scope: 0]
│           ├── Identifier: x --> [Type: integer | Scope: 0]
│           └─ Literal: 5 --> [Type: integer | Scope: 0]

```

Instruksi selanjutnya adalah melakukan perbandingan jika x lebih dari 5. x berada di address 3 sehingga sistem mengembangkan perintah untuk memuat value dari address 3. Sistem juga mengembangkan perintah untuk memanggil nilai literal yaitu 5. Setelah itu, untuk melakukan perbandingan, gunakan instruksi OPR ke-11 (GTR) untuk melakukan perbandingan x lebih besar dari 5. Hasilnya akan menjadi seperti ini.

.
.
.
.
.
.
5 - x
4 - Variable (y)
3 - Variable (x)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Push nilai x ke memori

.
.
.
.
.
.
6 - 5
5 - x
4 - Variable (y)
3 - Variable (x)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Push nilai literal 5 ke memori

.
.
.
.
.
.
4 - Variable (y)
3 - Variable (x)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Pop nilai x dan 5 untuk melakukan perbandingan GTR

Instruksi akan terlihat seperti ini

```
3 LOD 0 3
4 LIT 0 5
5 OPR 0 11
```

5. THEN-Statement (Assign $y = x*2$)

```

Then:
  Assign --> [Type: integer | Scope: 0]
    Identifier: y --> [Type: integer | Scope: 0]
    BinaryExpr: '*' --> [Type: integer | Scope: 0]
      Identifier: x --> [Type: integer | Scope: 0]
      Literal: 2 --> [Type: integer | Scope: 0]

```

Setelah itu, mari kita lewatkan bagian 'JPC' pada line '6' terlebih dahulu dan memeriksa bagian then terlebih dahulu. Tugasnya adalah memberi nilai y sebesar $x*2$. Fokus terlebih dahulu ke $x*2$, melihat instruksi ini. Urutan instruksi untuk sistem yang tepat adalah

- Memuat nilai x (LOD 0 3)
- Memuat nilai literal 2 (LIT 0 2)
- Ambil 2 nilai supaya bisa melakukan perkalian (OPR 0 4)
- Simpan nilai di y (STO 0 4)

Berdasarkan tahapan tersebut, langkah-langkah akan menjadi seperti ini

```

7  LOD  0  3
8  LIT  0  2
9  OPR  0  4
10 STO  0  4

```

6. ELSE-Statement (Assign $y = x/2$)

```
| | | | Else:  
| | | |   Assign --> [Type: integer | Scope: 0]  
| | | |     Identifier: y --> [Type: integer | Scope: 0]  
| | | |     BinaryExpr: 'div' --> [Type: integer | Scope: 0]  
| | | |       Identifier: x --> [Type: integer | Scope: 0]
```

```
| | | | | | | Literal: 2 --> [Type: integer | Scope: 0]
```

Pada kondisi else. Tugasnya adalah memberi nilai y sebesar $x/2$. Fokus terlebih dahulu ke $x/2$, melihat instruksi ini. Urutan instruksi untuk sistem yang tepat adalah

- Memuat nilai x (LOD 0 3)
- Memuat nilai literal 2 (LIT 0 2)
- Ambil 2 nilai supaya bisa melakukan pembagian (OPR 0 5)
- Simpan nilai di y (STO 0 4)

Berdasarkan tahapan tersebut, langkah-langkah akan menjadi seperti ini

```
12 LOD 0 3
13 LIT 0 2
14 OPR 0 5
15 STO 0 4
```

7. Statement lengkap IF-ELSE

```
| | | | | IfStatement --> [Type: if_statement | Scope: 0]
| | | | | | Condition:
| | | | | | | BinaryExpr: '>' --> [Type: boolean | Scope: 0]
| | | | | | | | Identifier: x --> [Type: integer | Scope: 0]
| | | | | | | | Literal: 5 --> [Type: integer | Scope: 0]
| | | | | | | Then:
| | | | | | | | Assign --> [Type: integer | Scope: 0]
| | | | | | | | | Identifier: y --> [Type: integer | Scope: 0]
| | | | | | | | | BinaryExpr: '*' --> [Type: integer | Scope: 0]
| | | | | | | | | | Identifier: x --> [Type: integer | Scope: 0]
| | | | | | | | | | Literal: 2 --> [Type: integer | Scope: 0]
| | | | | | | | Else:
| | | | | | | | | Assign --> [Type: integer | Scope: 0]
| | | | | | | | | | Identifier: y --> [Type: integer | Scope: 0]
| | | | | | | | | | BinaryExpr: 'div' --> [Type: integer | Scope: 0]
| | | | | | | | | | | Identifier: x --> [Type: integer | Scope: 0]
| | | | | | | | | | | Literal: 2 --> [Type: integer | Scope: 0]
```

Melihat statement-statement yang sudah dibuat, ini artinya 'JPC' memiliki tujuan ke line '12' karena disitulah lokasi 'else' didefinisikan. Setelah line '10' akan didefinisikan juga instruksi untuk lompat ke line '16' yaitu line setelah instruksi ELSE selesai. Oleh karena itu, instruksi blok-IF secara lengkapnya akan terlihat seperti ini

```

3 LOD 0 3
4 LIT 0 5
5 OPR 0 11
6 JPC 0 12
7 LOD 0 3
8 LIT 0 2
9 OPR 0 4
10 STO 0 4
11 JMP 0 16
12 LOD 0 3
13 LIT 0 2
14 OPR 0 5
15 STO 0 4

```

8. WRITELN

```

| | | | WriteStatement
| | | | | Expression:
| | | | | | Identifier: y --> [Type: integer | Scope: 0]

```

Bagian ini cukup sederhana. Panggil saja variabel yang akan dijadikan parameter yaitu y lalu panggil OPR ke-14 yaitu Writeln sesuai dengan daftar operasi yang ada di tabel sebelumnya

```

16 LOD 0 4
17 OPR 0 14

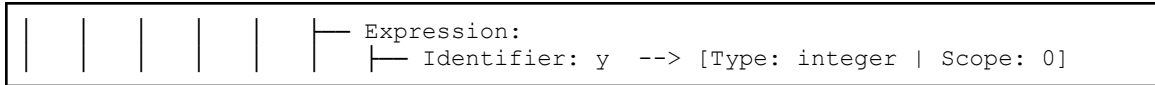
```

9. WHILE-Statement

```

| | | | WhileStatement --> [Type: while_statement | Scope: 0]
| | | | | Condition:
| | | | | | BinaryExpr: '>' --> [Type: boolean | Scope: 0]
| | | | | | | Identifier: y --> [Type: integer | Scope: 0]
| | | | | | | Literal: 0 --> [Type: integer | Scope: 0]
| | | | | Body:
| | | | | | CompoundBlock
| | | | | | | Assign --> [Type: integer | Scope: 0]
| | | | | | | | Identifier: y --> [Type: integer | Scope: 0]
| | | | | | | | BinaryExpr: '-' --> [Type: integer | Scope: 0]
| | | | | | | | | Identifier: y --> [Type: integer | Scope: 0]
| | | | | | | | | Literal: 1 --> [Type: integer | Scope: 0]
| | | | | | | WriteStatement

```



Berdasarkan decorated AST ini, kita bisa mengelompokkannya menjadi beberapa instruksi. Berikut adalah instruksi-instruksinya

- Melakukan pemeriksaan $y > 0$ awal. Jika sudah tidak memenuhi syarat langsung hindari While Statement
- Jika memenuhi, muat nilai y dan nilai 1 lalu lakukan operasi pengurangan
- Setelah itu, simpan di address-nya y
- Muat ulang nilai y lalu panggil operasi Writeln
- Lakukan pemeriksaan $y > 0$ dengan lompat kembali ke instruksi pemeriksaan $y > 0$ awal tersebut

Beginilah instruksinya dengan menggunakan Intermediate Code

18	LOD	0	4
19	LIT	0	0
20	OPR	0	11
21	JPC	0	29
22	LOD	0	4
23	LIT	0	1
24	OPR	0	3
25	STO	0	4
26	LOD	0	4
27	OPR	0	14
28	JMP	0	18

10. Closing

Lakukan return dengan menggunakan RET.

29 RET

Pada akhirnya, beginilah Intermediate Code secara full.

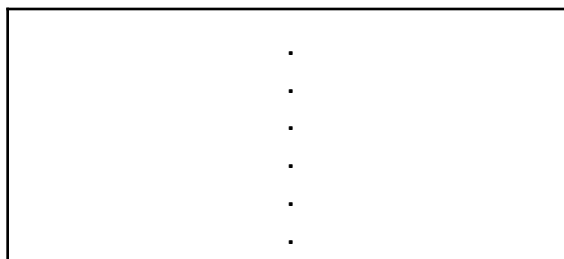
```
0 INT 0 5
1 LIT 0 10
```

```
2 STO 0 3
3 LOD 0 3
4 LIT 0 5
5 OPR 0 11
6 JPC 0 12
7 LOD 0 3
8 LIT 0 2
9 OPR 0 4
10 STO 0 4
11 JMP 0 16
12 LOD 0 3
13 LIT 0 2
14 OPR 0 5
15 STO 0 4
16 LOD 0 4
17 OPR 0 14
18 LOD 0 4
19 LIT 0 0
20 OPR 0 11
21 JPC 0 29
22 LOD 0 4
23 LIT 0 1
24 OPR 0 3
25 STO 0 4
26 LOD 0 4
27 OPR 0 14
28 JMP 0 18
29 RET
```

Berikut adalah step-by-step dari konversi Intermediate Code ke Interpreter dengan menggunakan Stack Machine

1. Inisiasi Memori

Inisiasi dilakukan dengan mengalokasikan memori sebesar 5. (Sisanya akan digunakan untuk operasi perhitungan, dll)



2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 1A: Inisiasi memori awal

· · · · ·
4 - Variable (y)
3 - Variable (x)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 1B: Inisiasi variabel

2. Simpan nilai 10 ke x

Pada bagian ini, nilai 10 akan ditambahkan ke *stack* machine lalu diambil lagi untuk disimpan ke variabel x di address 3.

· · · · ·

5 - Literal 10
4 - Variable (y)
3 - Variable (x)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 2A: Menambah nilai literal 10 ke *stack*

.
.
.
.
.
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 2B: Menyimpan nilai literal 10 ke address 3

3. IF-THEN-ELSE statement

.
.
.
.
.
.
6 - literal 5
5 - x
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 3A: Membandingkan nilai x dengan 5. Push nilai x dan 5

.
.
.
.
.
.
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 3B: Mengambil nilai x dan 5 untuk dibandingkan

x	>	5
---	---	---

Tahap 3C: Membandingkan jika nilai x lebih besar dari 5

.
.
.
.
.
.
6 - literal 2
5 - x
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 4A (Then): Push nilai x dan 2

.
.
.
.
.
.
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address

1 - Dynamic Link
0 - Static Link

Tahap 4B (Then): Mengambil nilai x dan 2 supaya bisa dilakukan perkalian

x	*	2
---	---	---

Tahap 4C (Then): x dikali 2

.
.
.
.
.
.
5 - $x*2$
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 4D (Then): Memasukkan hasil perkalian ke dalam stack

·
·
·
·
·
·
4 - Variable ($y = x * 2$)
3 - Variable ($x = 10$)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 4E (Then): Memasukkan nilai perkalian ke dalam memori variabel y

·
·
·
·
·
·
6 - literal 2
5 - x
4 - Variable (y)
3 - Variable ($x = 10$)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 5A (Else): Push nilai x dan 2

·
·
·
·
·
·
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 5B (Else): Mengambil nilai x dan 2 supaya bisa dilakukan pembagian

x	/	2
---	---	---

Tahap 5C (Else): x dibagi 2

·
·
·
·
·
·
5 - x/2
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link

0 - Static Link

Tahap 5D (Else): Memasukkan hasil pembagian ke dalam stack

.
.
.
.
.
4 - Variable ($y = x/2$)
3 - Variable ($x = 10$)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 5E (Else): Memasukkan nilai pembagian ke dalam memori variabel y

4. WRITELN

.
.
.
.
.
5 - y
4 - Variable (y)
3 - Variable ($x = 10$)
2 - Return Address

1 - Dynamic Link
0 - Static Link

Tahap 6A: Memasukkan nilai y ke dalam stack

.
.
.
.
.
.
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 6B: Mengambil nilai y supaya bisa dilakukan operasi yaitu WriteIn

20

Tahap 6C: Mengeluarkan output 20

5. WHILE statement

.
.
.
.
.
.
6 - literal 0
5 - y
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 7A: Membandingkan nilai y dengan 0. Push nilai y dan 0

.
.
.
.
.
.
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 7B: Mengambil nilai y dan 0 untuk dibandingkan

y	>	0
---	---	---

Tahap 7C: Membandingkan jika nilai y lebih besar dari 0

.
.
.
.
.
6 - literal 1
5 - y
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 8A (Failed to jump): Push nilai y dan 1

.
.
.
.
.
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address

1 - Dynamic Link
0 - Static Link

Tahap 8B (Failed to jump): Mengambil nilai y dan 1 supaya bisa dilakukan pengurangan

y	-	1
---	---	---

Tahap 8C (Failed to jump): y dikurangi 1

· · · · · ·
5 - y-1
4 - Variable (y)
3 - Variable (x = 10)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 8D (Failed to jump): Memasukkan hasil pembagian ke dalam stack

.
.
.
.
.
.
4 - Variable ($y = y-1$)
3 - Variable ($x = 10$)
2 - Return Address
1 - Dynamic Link
0 - Static Link

Tahap 8E (Failed to jump): Memasukkan nilai pembagian ke dalam memori variabel y

Tahap 9: Kembali ke tahap 7 untuk perbandingan. Jika berhasil, ke tahap 10

6. Return

Tahap 10 (Allowed to jump): Return

Bilamana masih ada kekeliruan dari tahapan-tahapan diatas, silahkan kunjungi guidebook pada link [ini](#).

IV. Teknis Pengerjaan

- Bahasa pemrograman yang digunakan **disamakan** dengan pengerjaan milestone sebelumnya.
- Pengerjaan dilakukan pada **repository yang sama** dengan milestone sebelumnya.
- **Jangan lupa menginvite asisten masing-masing ke dalam repository agar tugas dapat dinilai.**
- Anda **WAJIB** membuat **laporan** dengan format **Laporan-X-[KODE].pdf**, dengan **X** adalah **nomor Milestone** dan **Kode** adalah **Kode Kelompok**.
- **Laporan setidaknya berisi:**
 - Cover dengan foto kelompok menggantikan logo ITB.
 - Landasan Teori
 - Jelaskan teori-teori atau konsep yang berhubungan atau digunakan pada Milestone 4.
 - Perancangan & Implementasi
 - Implementasi yang ditambahkan dan berhubungan dengan Milestone 4.
 - Pengujian
 - Setidaknya 5 pengujian kasus unik.
 - Setiap pengujian harus menunjukkan hasil input/output program.
 - Bukti berupa screenshot input/output program.
 - Kesimpulan dan Saran
 - Lampiran berupa:
 - Link Release Repository Github.
 - Pembagian Tugas (menunjukkan persentase kontribusi).
 - Referensi yang digunakan selama pengerjaan Milestone 4
- Anda **WAJIB** membuat **laporan** dengan format **Laporan-X-[KODE].pdf**, dengan **X** adalah **nomor Milestone** dan **Kode** adalah **Kode Kelompok**.

- Jika ada pertanyaan lebih lanjut, silahkan ditanyakan pada [QNA](#).
- Link Pembagian Kelompok: [Sheets Pembagian Kelompok](#)

V. Penilaian

Program (70 poin)			
No	Komponen	Poin	Syarat Poin Maksimal
1.	Konversi Decorated AST ke Three Address Code	7	Program mampu mengubah decorated AST menjadi Intermediate Code
2.	Kesesuaian klasifikasi tipe, operand, dll berdasarkan Decorated AST	7	Program dengan tepat mengubah komponen decorated AST menjadi Three Address Code sesuai dengan informasi dari decorated AST
3.	Implementasi Memori Eksekusi	10	Program mampu menyimpan informasi decorated AST dengan membuat <i>stack</i> penyimpanan
4.	Struktur/modularitas program	8	Kode ditulis dengan struktur modular menggunakan fungsi atau kelas yang terpisah berdasarkan tanggung jawab. Alur eksekusi program mudah diikuti dan tidak redundan
5.	Error handling	5	Program dapat menangani error dengan baik (pesan informatif dan error tidak menyebabkan crash)
5.	Penerimaan input	3	Program dapat membaca decorated AST
6.	Penghasilan output	10	Program menghasilkan output Intermediate Code dan output final dari Source Code Arion dengan tepat
7.	Dokumentasi/komentar	5	Kode diberi komentar yang informatif dan tidak berlebihan
9.	Pengujian	15	Program berhasil lolos seluruh kasus uji

Laporan & Diagram (30 poin)			
No	Komponen	Poin	Syarat Poin Maksimal
1.	Penjelasan konsep	20	Penjelasan mengenai konsep intermediate code (Intermediate Code) dan arsitektur mesin eksekusi
2.	Perancangan program	5	Penjelasan mengenai rancangan program yang mencakup teknologi yang digunakan, struktur program, fungsi/kelas utama, dan alur kerja program. Terdapat pula justifikasi atas pilihan implementasi
3.	Hasil implementasi	2	Dokumentasi mengenai hasil implementasi, dapat berupa tangkapan layar dengan penjelasan
4.	Test case	3	Dokumentasi mengenai pengujian mandiri (minimal 5 dan mencakup berbagai kasus)

VI. Pengumpulan

- **Deliverables:**
 - Laporan
 - Diagram
 - Release Repository
- Pada setiap Milestone, Anda **WAJIB** mengumpulkan **Link menuju Release pada Repository Github.**
 - Gunakan format tag berupa **v0.X.Y** dengan **X** adalah **nomor milestone** dan **Y** adalah **nomor pengumpulan.**
 - Sebagai contoh, untuk milestone 1, release pertama, dan pengumpulan pertama adalah **v0.1.1**. Untuk pengumpulan selanjutnya karena revisi, gunakan tag **v0.1.2** dan seterusnya sesuai nomor pengumpulan. Untuk milestone selanjutnya, pengumpulan dimulai kembali dari satu yaitu **v0.2.1, v0.2.2**, dan seterusnya.
- **Laporan** dikumpulkan dengan cara **meletakkan** hasil **PDF** dalam **folder doc** dengan format **Laporan-X-[KODE].pdf**, dimana **X** adalah **nomor milestone**. Pastikan isi laporan sudah sesuai dengan [teknis pengerjaan](#).
- Link Pengumpulan: [Pengumpulan Milestone](#)
- Deadline milestone: **Kamis, 4 Juni 2026 pukul 23.59 WIB**

Referensi

"Compilers - Principles, Techniques, and Tools (2006)"

[https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20(2006).pdf)

Guidebook Konvensi Intermediate Code

☰ Lampiran Spesifikasi Milestone 4 - Tubes IF2224 TBFO - Guidebook Konvensi In...

Lampiran

Laman Pembagian Kelompok: [Pembagian Kelompok - Tubes IF2224 TBFO](#)

Laman QnA Tugas Besar: [QNA - Tubes IF2224 TBFO](#)

Guidebook Konvensi Intermediate Code:

[Lampiran Spesifikasi Milestone 4 - Tubes IF2224 TBFO - Guidebook Konvensi In...](#)

From Pengumpulan: [Form Pengumpulan - Tubes IF2224 TBFO](#)